Project Design Phase
Solution Architecture

Date	01 October 2026
Team ID	SWTID-2026-8643
Project Name	Script-Controlled ACL - Restrict Record Access Based on Field Value
Maximum Marks	4 Marks

Solution Architecture:

Solution architecture connects the business problem with an appropriate technology solution. Solution architecture bridges the gap between business problems and technology solutions. For this project, the For this project, the challenge is unrestricted access to student records, while the solution uses a layered, role-based and script-controlled ACL design on the ServiceNow platform. Its goals are to: problem is unrestricted access to student records, and the solution is a layered, role-based and script-controlled ACL design built on the ServiceNow platform. Its goals are to:

? Identify an effective technology solution for the business problem: restrict record access by field value (Branch) and by role. • Find the best tech solution to solve the business problem: restrict record access by field value (Branch) and by role.

? Explain the structure, characteristics and behavior of the solution to project stakeholders. • Describe the structure, characteristics and behavior of the solution to project stakeholders.

? Define the features, development stages and solution requirements. • Define features, development phases and solution requirements.

? Provide specifications through which the solution is defined, managed and delivered. • Provide specifications according to which the solution is defined, managed and delivered.

Solution Architecture Diagram:

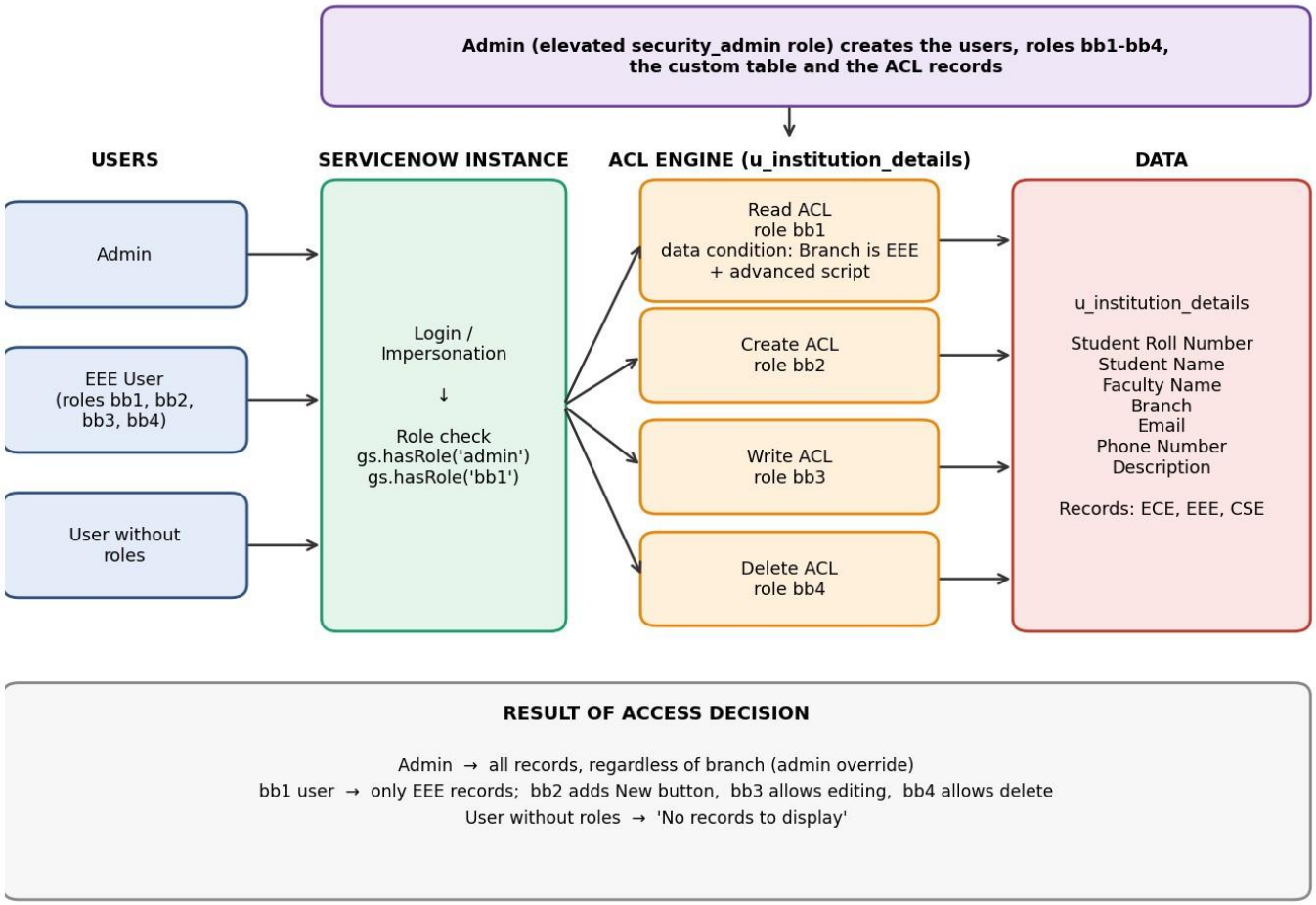


Figure 1: Architecture and data flow of the script-controlled ACL solution

Components

Component	Description
Users and Roles	EEE User with custom roles bb1 (Read), bb2 (Create), bb3 (Write) and bb4 (Delete); Admin with the admin role (and security_admin elevated for ACL setup).
Custom Table	u_institution_details with fields Student Roll Number (auto number), Student Name, Faculty Name, Branch (ECE, EEE, CSE), Email, Phone Number and Description.
ACL Engine	Four record-type ACLs on the table, one per operation (read, create, write, delete). The Read ACL is advanced: data condition 'Branch is EEE' plus a script using gs.hasRole().
Verification	Impersonation of the EEE user, a user without roles, and Admin, followed by opening the table list (u_institution_details.list).

Data Flow

1. Admin elevates to security_admin and creates the users, roles bb1-bb4, the table u_institution_details and sample records.. Admin elevates to security_admin and creates the users, roles bb1-bb4, the table u_institution_details and sample records.
2. Admin creates the Read, Create, Write and Delete ACLs, with each ACL requiring its assigned role.. Admin creates the Read, Create, Write and Delete ACLs, each requiring its own role.
3. A user signs in (or is impersonated) and opens the table list.. A user logs in (or is impersonated) and opens the table list.
4. ServiceNow checks the user's roles with gs.hasRole() and evaluates the ACL for each operation.. ServiceNow checks the user's roles with gs.hasRole() and evaluates the ACL for each operation.
5. The Read ACL checks the record's Branch value against the data condition (Branch is EEE) and executes the advanced script.. The Read ACL checks the record's Branch value against the data condition (Branch is EEE) and runs the advanced script.
6. The result is returned: permitted records are shown, while denied records remain hidden from the list and form.. The result is returned: allowed records are shown and denied records are hidden from the list and form.

Access Matrix

User / Role	Read	Create	Write	Delete
Admin	All records	Yes	Yes	Yes
bb1	Only EEE records	No	No	No
bb1 + bb2	Only EEE records	Yes (New button)	No	No
bb1 + bb2 + bb3	Only EEE records	Yes	Yes (edit)	No
bb1 + bb2 + bb3 + bb4	Only EEE records	Yes	Yes	Yes
No roles	No records to display	No	No	No

Reference: ServiceNow Access Control (ACL) documentation, System Security module.